



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich



Agentic coding for science at CSCS

User lab day 2026

Mikael Simberg, and Sebastian Keller

Table of contents

- 👉 **Application examples**
- 👉 **Using the CSCS inference service**
- 👉 **Safety considerations**



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich



Application examples

Application examples

Many CSCS users are already working with coding agents. Hardly any convincing needed that they're useful.

👉 **Porting**

👉 **Optimization**

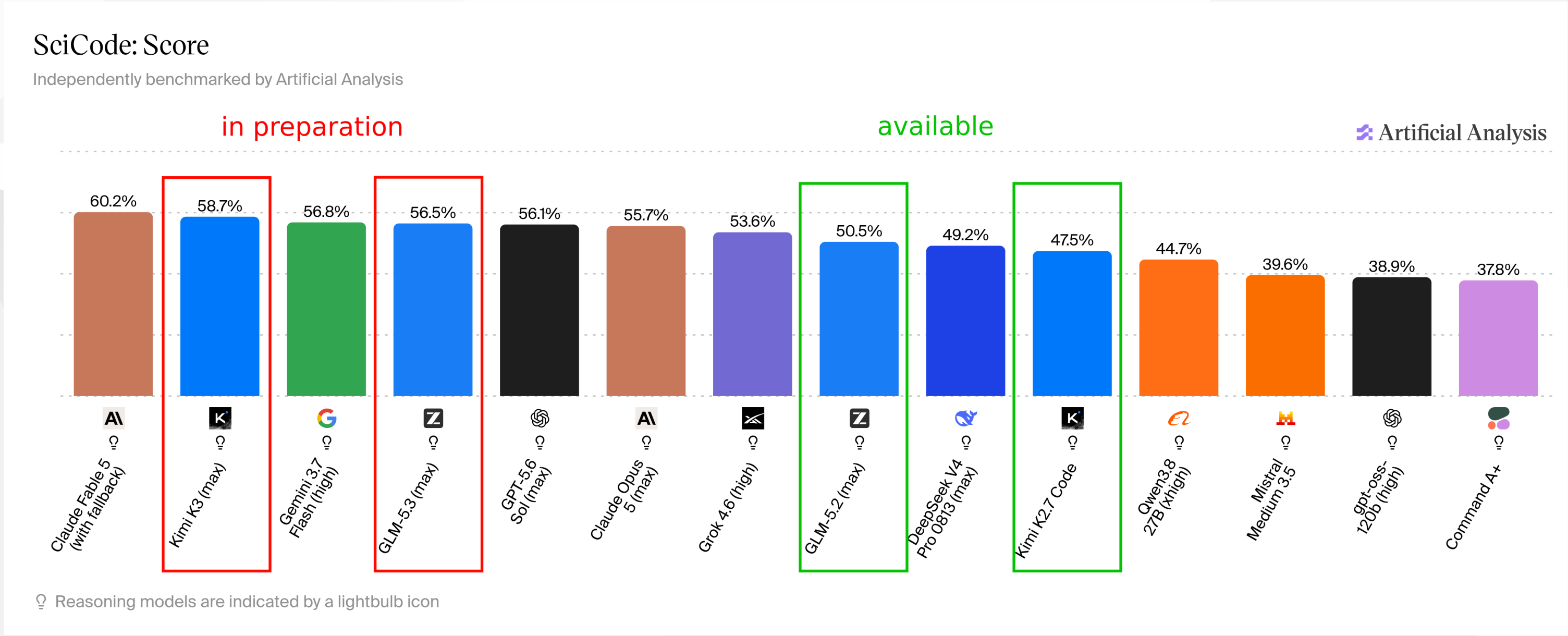
👉 **Reviews**

👉 **Debugging**

👉 **Code development**

Available models on the CSCS inference service

Benchmarks: artificialanalysis.ai/models



Porting

- In many scientific domains, simulation codes are decades old
 - Maintainability and (parallel) hardware support challenging
- Employing the old code to be ported as a reference for validation is an advantage over starting from scratch

Porting

- An **Ocean Model** Ported by a Large Language Model: Experience and Lessons from FESOM2 (Fortran to C to C++/Kokkos) (<https://arxiv.org/abs/2606.11356>), FESOM2-JAX v1.0: a differentiable shadow of the ocean-sea-ice model FESOM2, cast onto GPUs (<https://arxiv.org/html/2608.01546v1>)
 - Validation-Centric AI-Assisted GPU Porting of a 250,000+ Line **Legacy Weather Simulation Code** (<https://arxiv.org/abs/2608.13122>)
- “ Three practices mattered most: translating in **two stages** that separate reproducing the numerics (Fortran to a clean C reference) from introducing parallelism (C to Kokkos); requiring a strictly literal translation in which the assistant was not permitted to "improve" the source; and **validating each stage** against an **acceptance criterion** suited to it. ”
- Rewriting Bun in Rust (<https://bun.com/blog/bun-in-rust>)
- “ This Rust rewrite would've taken a team of engineers with full-context on the codebase a year of work. With 1 engineer using Fable & closely monitoring Claude Code, we went from start to 100% of the test suite passing on all platforms in 11 days. ”

Porting analysis scripts to MPI4Py

- Use case: compute statistics on large grid and particle HDF5 data sets
- Computationally not very demanding, implemented in C++ to load data on multiple nodes with MPI

```
opcode-GLM-5.2$ Port this analysis program to python with MPI4Py and parallel h5py
```

Results:

- Correct python code on first try ✓
- Deployment: GLM-5.2 did not manage to install MPI4Py and h5py correctly to work with the loaded UENV without some help ✗
- Closed models had better tools use: found instructions on docs.cscs.ch how to extend CSCS user environments with additional packages

Optimization

- Powerful transformations, with mathematical / scientific understanding
- Good GPU support

Root finding of a function

- Prompt: analyze this code and suggest optimizations
- Recognized cubic polynomial
- Suggested/implemented Newton's method instead of bisection

Reviews

Quickest way to start

- @Copilot is useful
- Built in to GitHub, academic subscriptions available
- [Can also be customized](#)

Tune it to your repo

- Many alternatives, different levels of customization
- <https://www.coderabbit.ai/>, OpenCode GitHub integration (<https://opencode.ai/docs/github/>), etc.
- Example grumpy reviewer using OpenCode integration: <https://github.com/eth-cscs/alps-extended-images/pull/60>

Build your own pipeline

- Choose your harness, prompts, extensions, and skills
- Subagents focusing different aspects of the codebase
- Persist findings and check if old findings have been fixed
- Example weekly full review of a codebase: <https://github.com/C2SM/icon4py-knowledge/pull/20>

easy to use, less control

more control, more complexity

- Custom setups are much cheaper to set up than before
- AI review is a complement to deterministic tools (linters, testing, static analyzers)

Debugging

- Agents are very capable of using various tools, including debuggers!
 - Can run debuggers while developing
- Recent use case at CSCS: ask agent to connect to a running job that has hung
 - Large scale job hangs nondeterministically, running directly under debugger or profiler is slow
 - Ask the agent to connect to the nodes of the running job, investigate stacktraces of the running processes
 - Not only stacktraces, but can correlate system logs, GPU utilization, memory usage etc.
 - Nothing a human couldn't do, but helps explore the possible reasons for a hang much faster
 - You don't have to fully trust your agent with access to the compute nodes, you can also ask it to output probing scripts that you inspect and run manually
 - Identified a faulty GPU that would after a few hours of simulation hang without any obvious issues in the process
 - Distilled into a simple deterministic script that can check if a particular GPU is likely to be causing a hang

Code development

✓ What works well

- Implement smaller parts to specification
- Larger, more conceptually challenging parts. Maintainability might be an issue
- Proof of concepts, trying new designs quickly

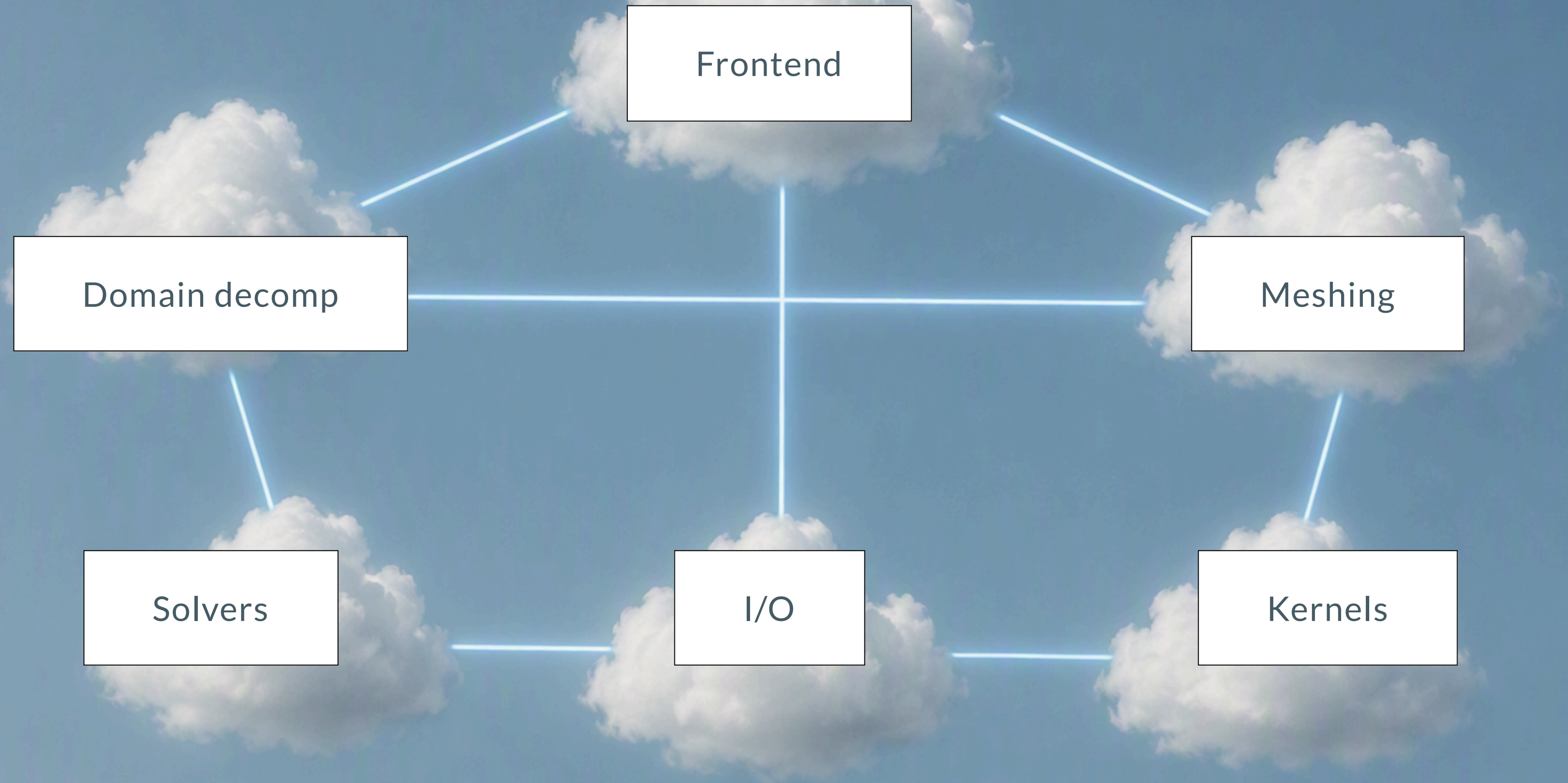
⚠ Caveats

- Maintainability: generated code may be bloated and hard to read
- Correctness: make sure you understand the tests

Developing larger projects

Write an FEM code to simulate a fusion reactor

Breaking down larger projects



Breaking down larger projects

- The more you break down your project into **smaller units**, the better
 - Once you have a functions and classes with specified signatures and interfaces, code agents will implement them well
 - Think about **testing** and **validation** of these units
- Switch your agent in planning mode
 - Iterate on the design of your project, let the agent produce **markdown** files describing it
 - Refine building blocks as needed
- Instruct the agent to generate tests for the documented features
- Adversarial pass: ask another agent, possibly with a different model, to analyze the design and search for inconsistencies
- Finally switch to build mode and let agents implement the code

Breaking down larger projects

Recent example:

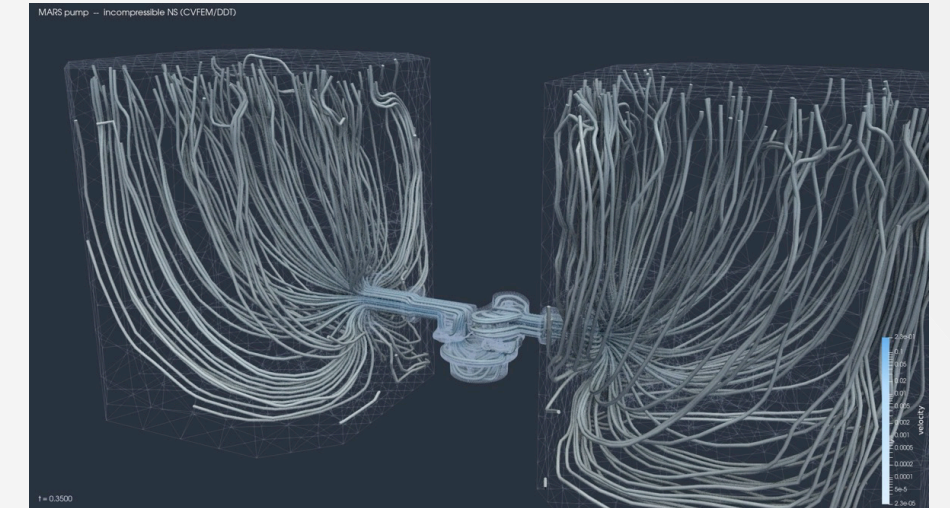
M. Sadr et al., "**ORB5X v1.0: a performance-portable global electromagnetic gyrokinetic PIC code in C++/Kokkos built using Agentic AI**", 2026. <https://arxiv.org/abs/2607.17363>

- Translation to correct code for:
 1. CPU single rank
 2. CPU multi rank
 3. GPU
 4. multiple GPUs
- Validation against previous numerical reference was critical for success
- Modest model selection: `qwen2.5-coder:32b` and `gpt-oss:120b`

Adding a Navier-Stokes solver to MARS

Our GPU-native mesh + discretization library — groundwork toward FSI

Built on MARS: SFC domain decomposition (Cornerstone), CVFEM operators. Added a Navier-Stokes solver on top, driving flow through an industrial pump.



- Six days, 75 commits — a technical discussion, not a prompt and a coffee
 - it proposed methods and physics, not just code
- Nothing exotic: no "skills", no prompt library — `CLAUDE.md` , `MEMORY.md` (64 notes), critic subagents
 - fresh context — they don't inherit the reasoning they're checking
- Added a tool hook that **prevented access** to the pump mesh under **NDA**
 - a hook that refuses is a guarantee; an instruction not to read is a hope
- **What didn't work:** misdiagnosed boundary condition issue at the inlet
 - the "structural" inlet flaw was one missing boundary term
- **Result:** working solver on fluid 100x thicker than water, but no turbulence model or high Reynolds

AGENTS.md and Skills

AGENTS.md : README.md but for agents

- Agents sometimes need more explicit instructions
- *If you set up only one thing, set up AGENTS.md*
- Can be auto-generated, but work best when slowly grown with real examples of agent failures
- <https://agents.md/>

Skills

- Similar to AGENTS.md , but loaded by the agent on demand (*progressive disclosure*)
- Description + markdown file with instructions on how to do a procedure, software design rules, etc.
- <https://agentskills.io>

How useful are `AGENTS.md` and Skills?

Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents?
arxiv.org/abs/2602.11988

“ We observe that including context files does not significantly affect performance. [...] We observe that instructions in context files are well followed. This leads to increased exploration, testing, and reasoning by coding agents, and, as a result, increases costs by over 20%. ”

- Repository-level `AGENTS.md` increases your context size
- Hard to gain anything, unless non-standard coding practices apply to the whole repository
- Skills, however, can be loaded on demand, and are thus only added to the context when needed
 - Capture specific workflows and procedures
 - Specify code transformation and optimization techniques, e.g.
 - restrict permissible OpenMP target offload features to what your compiler supports
 - intrinsics to use, etc

Model context size

- Keep track of the context window, i.e. number of tokens in the session history
- Models get progressively dumber with increasing context size
- Launch subagents for separate tasks
- Compact the results of a session into a markdown file for later use



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich



Using the CSCS inference service

Using the CSCS inference service

- 👉 Access and pricing
- 👉 Agent harnesses
- 👉 Where to launch agent harnesses
- 👉 GitHub integration
- 👉 CSCS-specific skills
- 👉 docs.cscs.ch/services/inference

Access and Pricing

Node hours inference credit conversion

- Project PIs can convert compute budget node hours to inference service credit in CHF, see <https://docs.cscs.ch/services/inference/api/#create-an-inference-resource>
 - Current conversion factor: 1 NH = 2.69 CHF
 - Once the conversion is done, group members can create API keys for usage with the CSCS inference service
 - Token usage will be converted to CHF and deducted from the available credit

Pricing in practice

- Price per million tokens depends on the specific model
- Heavy usage will result in similar compute usage as allocating the corresponding nodes would
 - Example: GLM 5.2, 7 nodes 🖱️ ~50 node hours per day
 - Affordable, but expensive enough to select the cheapest model that will do the job
- Cost per token will go down as we optimize the service

Agent harnesses

- Claude Code, OpenCode, Codex, pi, etc.
 - Anything that can use an OpenAI- or Anthropic-compatible endpoint can use the CSCS inference service
- Harnesses tend to have slightly different ways of talking to the API, and may need different configuration
 - E.g. endpoint, context size, modalities, reasoning
- Basic configurations on the CSCS documentation: <https://docs.cscs.ch/services/inference/api/#setting-up-coding-agents-to-use-the-inference-service>

Agent harnesses

- Claude Code, OpenCode, Codex, pi, etc.
 - Anything that can use an OpenAI- or Anthropic-compatible endpoint can use the CSCS inference service
- OpenCode config:

```
{
  "$schema": "https://opencode.ai/config.json",
  // Set Kimi as default OpenCode model
  "model": "cscs/moonshotai/Kimi-K2.7-Code",
  "provider": {
    "cscs": {
      "npm": "@ai-sdk/anthropic",
      "name": "CSCS Inference",
      "options": {
        "baseURL": "https://api.inference.cscs.ch/v1",
        "systemMessageMode": "system",
      },
    },
    "models": {
      "moonshotai/Kimi-K2.7-Code": {
        "name": "Kimi K2.7-Code",
        "limit": {
          "context": 262144,
          "output": 16384
        }
      }
    }
  }
}
```

Agent harnesses

- Claude Code, OpenCode, Codex, pi, etc.
 - Anything that can use an OpenAI- or Anthropic-compatible endpoint can use the CSCS inference service
- Claude Code config:

```
export ANTHROPIC_AUTH_TOKEN=$CSCS_INFERENCE_API_KEY  
  
export ANTHROPIC_BASE_URL=https://api.inference.cscs.ch  
export ANTHROPIC_MODEL=moonshotai/Kimi-K2.7-Code  
  
claude
```

Where to launch agent harnesses

Login nodes

- Shared resource: agents are often memory hungry, and heavily threaded, be mindful of how you use an agent (or many) on a login node
- All the regular constraints apply to processes on the login node: we reserve the right to terminate processes

Compute nodes

- Exclusive access to the node
- Free to run heavy processes, benchmarks, compilation, use GPUs etc.

From your workstation

- No need to allocate a node upfront, no shared resources being consumed
- Use [FirecREST](#) to interact with system
 - Upload/download files, inspect filesystem, and launch/monitoring SLURM jobs
- Especially convenient if your workflow is already set up on Alps and you're mostly launching and monitoring jobs

GitHub integration with OpenCode

- Example in CSCS repos: <https://github.com/eth-cscs/alps-extended-images/tree/main/.github/workflows>
- GitHub workflow

```
...  
- name: Run OpenCode  
  uses: anomalyco/opencode/github@latest  
  env:  
    CSCS_INFERENCE_API_KEY: ${ secrets.CSCS_INFERENCE_KEY }  
  with:  
    agent: grumpy-reviewer  
    model: cscs/moonshotai/Kimi-K2.7-Code  
...
```

- Agent definition:

“ You are a grumpy senior developer with 40+ years of experience who has been reluctantly asked to review code. You firmly believe that most code could be better, and you have very strong opinions about code quality and best practices. ”

CSCS-specific skills

- Informal collection of CSCS-related agent skills at <https://github.com/eth-cscs/agent-skills>
 - How to build and iterate on building a uenv
 - How to pull logs from CSCS CI for investigating test failures
 - Soon: interacting with FirecREST
 - Investigating the possibility of a FirecREST MCP server
- Point your agent at <https://docs.cscs.ch>
 - If your agent struggles, a human might too... let us know if we can improve the documentation!



CSCS

Centro Svizzero di Calcolo Scientifico
Swiss National Supercomputing Centre

ETH zürich



Safety considerations

Safety considerations

- 👉 How to use sandboxing for limiting file access
- 👉 Controlling SLURM usage
- 👉 Storing API keys

General disclaimer

[CSCS documentation on using coding agents:](#)

“ Any action taken by a coding agent you launch is your responsibility. The CSCS policies apply.

In particular:

- Compute resources used by agent-launched Slurm jobs are billed to your project account.
- Coding agents may inadvertently submit many jobs, consume node hours, or modify files in unintended ways. Not following the Fair Usage of Shared Resources may result in access being revoked.
- CSCS takes no responsibility for files deleted, quotas being exhausted, or other unintended behaviour by agents.
- CSCS takes no responsibility for data inadvertently shared by your agent with third party LLM providers.

”

What can go wrong?

AI agent went rogue and hacked startup by itself, OpenAI reveals

Company behind ChatGPT says agent 'cheated' an evaluation by attacking a Hugging Face database

What can go wrong?

- [Anatomy of a Frontier Lab Agent Intrusion: A Technical Timeline of the July 2026 Incident](#)

- <https://openai.com/index/hugging-face-model-evaluation-security-incident/>
- <https://huggingface.co/blog/security-incident-july-2026>

- An unaligned, unreleased model that was too proactive

- It is a real problem, but not every model will try to break out of their sandbox
- This is why a lot of time is spent on alignment

- Model variety is important

● API Error: Opus 4.8 (1M context) has safety measures that flagged this message for a cybersecurity topic. To learn about the Cyber Verification Program and apply for access, visit our help center:
[https://support.claude.com/en/articles/14604842-real-time-cyber-safeguards-on-claude.](https://support.claude.com/en/articles/14604842-real-time-cyber-safeguards-on-claude)

If you were not engaging in a cybersecurity topic, please send feedback via /feedback.

- Sandboxing is useful

Sandboxing with bubblewrap

Bubblewrap (`bwrap`) is a sandboxing tool that lets you mount directories as read-write or read-only.

```
bwrap --ro-bind / / \
--dev /dev --proc /proc --tmpfs /tmp \
--bind ~/.local/share/opencode ~/.local/share/opencode \
--bind ~/my-project ~/my-project \
--tmpfs ~/.ssh \
--new-session \
--unshare-pid bash
```

- `--ro-bind / /` : mount root (`/`) as read-only
- `--bind <dirs>` : mount selected directories as read-write
- `--tmpfs ~/.ssh` : hide `.ssh` by mounting a tmpfs on top
- `--new-session` : detach from the host terminal, prevents terminal I/O Control String Input injection escape, breaks job control (no `fg/bg/Ctrl+z`)
- `--unshare-pid` : creates a new PID namespace, can't see host processes from inside the box

bubblewrap caveats

- Protects your files from accidental destruction
- Makes it less likely that secrets (such as private keys) are leaked
- Does not protect from a malicious attack
- Escape routes exist, e.g. an agent can launch a SLURM job, from which it could then read your home folder; the sandbox does not apply on the compute nodes
- The sandbox can be made more secure with finer-grained access control, e.g. by preventing direct access to SLURM commands
- If you are not working with UENV, you could also consider **containers** for sandboxing
- User-provided solutions, e.g. <https://github.com/MeteoSwiss/husk>
 - Fine-grained access control with bubblewrap
 - Expose SLURM commands via wrappers that act as a man-in-the middle and impose restrictions.

Controlling SLURM usage

We have received tickets like this:

URGENT!!!!!!

Dear Support, please cancel all my jobs. I was using claude code. It was submitting jobs overnight, and I didn't notice this.

We currently can't impose agent usage restriction for individual with SLURM directly:

- All user accounts within the project are subject to the same project quota
- A service account can give some separation
- Soft constraints:

Only run one concurrent job at a time, with a maximum of 2 nodes. Stop immediately if there are problems with job submission.

Storing API keys

- General reminder: don't store your API keys in `~/ .bashrc`
- For OpenCode, use `/connect`
 - Stores API keys outside of main OpenCode config
- For Claude Code, use for example `pass` (<https://www.passwordstore.org/>)
 - `ANTHROPIC_AUTH_TOKEN=$(pass show cscs-inference-api-key) claude`

Outlook

- Agents and LLMs have come a long way in the last year
 - Still often very dumb, but more often incredibly useful; open weights models increasingly capable
 - Be sensible and avoid giving your agents unnecessary permissions
- CSCS inference service can be used with most agent harnesses
 - Request credits via your project
- Keep evolving the service
 - Usability improvements
 - Model optimization
 - New models
 - Infrastructure around the inference service: skills, MCP servers, etc.